CLOUD SECURITY PATH · STUDY GUIDE 4

Cloud Engineering Sprint Workbook

Step-by-step tasks for every Phase A sprint (26 Oct 2026 – 28 Feb 2027): a secure AWS account, IAM, VPC, compute, storage, Terraform, serverless, containers, cloud logs into your Wazuh SIEM, and Azure. Then the outline for Phase B, cloud security (Mar – Dec 2027). Built for 20+ hours a week.

PHASE A · CLOUD ENGINEERING (2-week sprints)

S1	S2	S3	S4	S5	S6	S7	S8	S9
Launch	IAM + VPC	EC2 + S3	Terraform	Holiday	Serverless	Containers	Azure	Capstone

PHASE B · CLOUD SECURITY (monthly sprints)

Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Identity	Data	Network	Posture	Detect	Respond	Containers	Sentinel	Capstone	Job sprint

Certs: Cloud Practitioner (Dec, optional) · **Solutions Architect Associate (early Mar)**
Security+ (May) · **AWS Certified Security – Specialty (Oct)**
Grey = lighter sprint (holidays, catch-up). Blue = Azure focus.

Contents

0 How to use this workbook
S1 Launch: SOC wrap-up + secure AWS account
S2 IAM + VPC by hand
S3 EC2, S3 + a secure static site
S4 Terraform: everything as code
S5 Holiday sprint
S6 Serverless, databases + Cognito
S7 Scaling, containers + CloudTrail → Wazuh
S8 The Azure sprint
S9 Phase A capstone
B Phase B: cloud security outline
R Resources
A Answers to the quick checks

0 How to use this workbook

> **Every sprint has one goal, a list of things to learn, a build, a cost warning, a deliverable for GitHub and a quick check. Finish the "done when" list, not every bullet.**

The sprint routine

1. **Day 1, plan (30 min).** Read the sprint. Copy its "done when" list to the top of `progress-log.md`.
2. **Learn just enough, then build.** Read the guide chapters, then go hands-on the same day. Building is where it sticks.
3. **Break it on purpose.** Remove a route, deny a permission, then fix it. Troubleshooting is the skill employers pay for.
4. **Write it up as you go:** what you built, why each setting matters for security, what broke and how you fixed it. Redact account IDs, emails and keys in screenshots.
5. **Last day, review.** Push to GitHub, then do a quiz and explain-back session with Claude. Add a three-line retro: worked / didn't / change.
6. **Destroy and check billing.** Every session ends with nothing expensive left running.

Your week (about 21 hours)

Block	Hours	What
Build sessions	8	4 × 2 h hands-on
Study	5	This workbook, Guide 3, official docs, a course
Exam practice	2	Practice questions for the current cert
SOC thread	3	One LetsDefend alert a day + TryHackMe SOC Level 1
Write-up + push	2	Notes, screenshots, GitHub
Sprint review	1	Quiz and explain-back with Claude

REMEMBER

Take one full rest day each week. Twenty focused hours beat thirty tired ones.

Why AWS first, Azure second

AWS's free plan gives you room to build, and most cloud engineering concepts carry over: a VPC is a VNet, a security group is an NSG, IAM roles are managed identities. Guide 3 maps every AWS service to its Azure twin, and S8 rebuilds the basics in Azure so you're comfortable in both by February.

How much it costs: the rules

Watch out for	Why	Habit
NAT gateway	Billed per hour plus per GB, even when idle	Use VPC endpoints or destroy after the session
Load balancers (ALB)	Billed per hour	Only during S7 and S9 sessions
RDS databases	Billed per hour while running	Smallest size; destroy after use
WAF	Monthly fee per web ACL and rule (charged by the hour)	Delete the web ACL when the sprint ends
Interface VPC endpoints	Small hourly fee each	Destroy with Terraform
Free trials (GuardDuty, Security Hub, Macie, Sentinel)	Paid after the trial ends	Start them in the month you use them (Phase B)

Check current prices on the AWS and Azure pricing pages; they change.

S1 Launch: SOC wrap-up + a secure AWS account

26 Oct – 8 Nov · Guides: G1 subnetting refresher; G3 ch. 1–5

> **Goal: close Lab Phase 5 with network logs in Wazuh, and open an AWS account the way a security professional would.**

Part 1: pfSense logs into Wazuh (Case 004)

1. **Wazuh: listen for syslog.** On the Wazuh server, add a second `<remote>` block inside `/var/ossec/etc/ossec.conf` (keep the existing one for agents). Use nano, and back up the file first.

```
sudo cp /var/ossec/etc/ossec.conf /var/ossec/etc/ossec.conf.bak2
sudo nano /var/ossec/etc/ossec.conf
  <remote>
    <connection>syslog</connection>
    <port>514</port>
    <protocol>udp</protocol>
    <allowed-ips>10.0.3.1</allowed-ips>
  </remote>
sudo systemctl restart wazuh-manager
```

`allowed-ips` means only pfSense's MGMT address may send logs: least privilege again.

2. **pfSense: send logs.** Status → System Logs → Settings → enable **Remote Logging**. Source address: MGMT. Remote server: `10.0.3.10:514`. Contents: Firewall events.
3. **Generate traffic (your own lab only).** From Kali, scan the DMZ server on ports the firewall blocks: `nmap -sS -p 1-1000 10.0.2.10`. `-sS` is a SYN scan: it sends the first packet of the handshake and never finishes it.
4. **Find it in Wazuh.** Search for `rule.groups: pfsense`. Wazuh's built-in pfSense rules flag blocked packets and many blocks from one source. If nothing decodes, paste a raw log line into **Ruleset Test** (test before you fix). If it still fails, set pfSense's log message format to BSD (RFC 3164) and test again.
5. **Write Case 004:** hypothesis → logs → rule → test → what a real analyst would do next.

Part 2: a secure AWS account

1. **Sign up** and pick the free plan. Choose one region and stay in it, for example **eu-west-2 (London)**.
2. **Root user:** turn on MFA, then sign out. The root user is only for billing and emergencies.
3. **Billing:** create a **zero-spend budget** and a **$5 budget** with email alerts.
4. **IAM Identity Center:** enable it, create a user for yourself with MFA, and give it the `AdministratorAccess` permission set. This is your day-to-day login: short-lived credentials, no long-term access keys.
5. **CloudTrail:** check that event history works, then create a trail that writes to an S3 bucket. You'll connect it to Wazuh in S7.
6. **AWS CLI on Kali**, using your Identity Center login:

```
aws configure sso          # follow the prompts; name the profile "lab"
aws sts get-caller-identity --profile lab   # "who am I?": shows account and role
```

IN YOUR LAB

Set `export AWS_PROFILE=lab` in `~/.bashrc` so you don't have to type `--profile` every time.

Learn this sprint

- Regions, availability zones, edge locations (G3 ch. 5)
- The shared responsibility model applied to AWS (G3 ch. 4)
- Root user vs IAM user vs Identity Center user; why long-term keys are risky

REMEMBER

Deliverable: Case 004 (pfSense scan detection) + `docs/phase-6-aws-account.md` with redacted screenshots of MFA, budgets and CloudTrail.

QUICK CHECK (answers at the back)

1. Why should you never use the root user for daily work?
2. What does `allowed-ips` do in the Wazuh syslog block, and why does it matter?
3. What is the difference between a region and an availability zone?

Interview question: "You've just been given a new AWS account. What are the first five things you do?"

S2 IAM + VPC by hand

9 – 22 Nov · Guides: G3 ch. 8, 10

> **Goal: understand identity and networking well enough to build them without a wizard. Everything else in the cloud sits on these two.**

Part 1: IAM

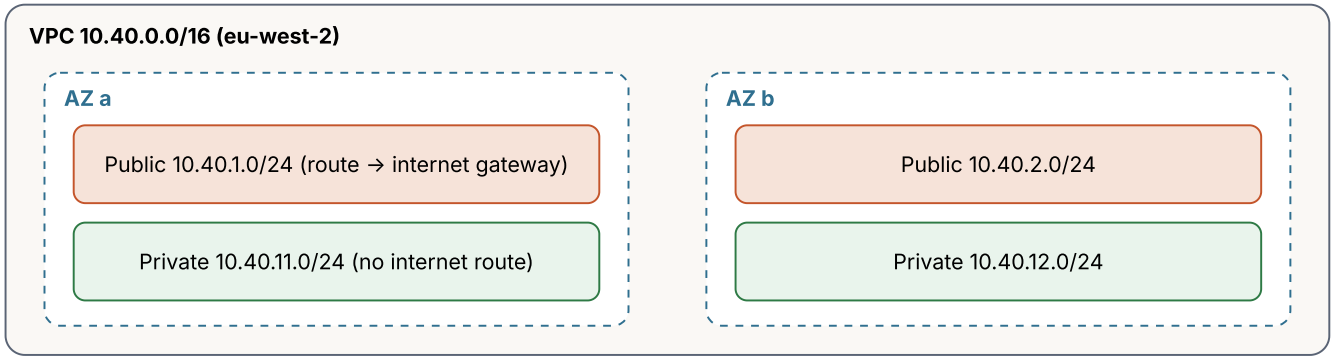
Term	Meaning
Principal	Who is acting: a user, a role, a service
Policy	A JSON document that allows or denies actions on resources
Role	An identity with no password that something *assumes* to get temporary credentials
Explicit deny	Always wins. Then allow. Anything not allowed is denied by default.

Build: write a policy that allows reading one S3 bucket only, attach it to a role, and test it. Explain every line in your write-up.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::my-lab-bucket",
                 "arn:aws:s3:::my-lab-bucket/*"]
  }]
}
```

`ListBucket` applies to the bucket itself; `GetObject` applies to the objects inside it (`/*`). Mixing those up is a classic mistake. **Break it:** remove one ARN and see which command fails.

Part 2: a VPC by hand



1. **In the console:** create the VPC, four subnets, an internet gateway, a public route table (`0.0.0.0/0 → igw`) and a private route table (local only). Associate each subnet with the right table.
2. **Then rebuild it with the CLI**, which forces you to understand each piece:

```
aws ec2 create-vpc --cidr-block 10.40.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=lab-vpc}]'
aws ec2 create-subnet --vpc-id vpc-XXXX --cidr-block 10.40.1.0/24 \
  --availability-zone eu-west-2a
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --vpc-id vpc-XXXX --internet-gateway-id igw-XXXX
aws ec2 create-route-table --vpc-id vpc-XXXX
aws ec2 create-route --route-table-id rtb-XXXX \
  --destination-cidr-block 0.0.0.0/0 --gateway-id igw-XXXX
aws ec2 associate-route-table --route-table-id rtb-XXXX --subnet-id subnet-XXXX
```

THINK OF IT LIKE THIS

A subnet is "public" only because its route table sends `0.0.0.0/0` to the internet gateway. Nothing else makes it public. It's the same idea as your pfSense zones: the routes and rules decide what can reach what.

	Security group	Network ACL
Applies to	An instance's network interface	A whole subnet
State	**Stateful** (like pfSense)	Stateless: you must allow replies yourself
Rules	Allow only	Allow and deny, numbered order

REMEMBER

Deliverable: `docs/phase-6-iam-vpc.md` with your policy explained line by line, a VPC diagram, your CLI commands with comments, and one thing you broke and fixed. Then delete everything.

QUICK CHECK (answers at the back)

1. What makes a subnet public?
2. Why does the policy above need two resource ARNs?
3. Security groups are stateful. What does that mean for return traffic?

Interview question: "Walk me through how a request from the internet reaches an instance in a public subnet."

S3 EC2, S3 + a secure static site

23 Nov – 6 Dec · Guides: G3 ch. 6–7

> **Goal: run a server with no public IP and no SSH, and publish a website without making its bucket public.**

Part 1: EC2 in a private subnet with Session Manager

1. Create an **IAM role for EC2** with the managed policy `AmazonSSMManagedInstanceCore`.
2. Launch a small Amazon Linux instance in a **private** subnet, with no public IP, no key pair, a security group with **no inbound rules**, and the role attached.
3. The instance can't reach the internet, so add **interface VPC endpoints** for `ssm`, `ssmmessages` and `ec2messages`. Their security group must allow HTTPS (443) from the VPC.
4. Connect from Kali (install the Session Manager plugin first): `aws ssm start-session --target i-XXXX`

REMEMBER

No open ports, no keys to steal, and every session is logged. This is how mature teams reach servers. It also answers "why no SSH?" in interviews.

Part 2: secure static site (S3 + CloudFront + WAF)

1. **S3 bucket** with **Block Public Access on**. Upload a simple `index.html`.
2. **CloudFront distribution** with the bucket as origin, using **Origin Access Control (OAC)**. CloudFront gives you a bucket policy that lets only that distribution read the bucket.
3. **HTTPS only:** viewer protocol policy "Redirect HTTP to HTTPS".
4. **WAF:** attach a web ACL to the distribution with the AWS managed *Common Rule Set* and a **rate-limit rule**. WAF for CloudFront lives in `us-east-1`, a common surprise.
5. **Prove it:** the bucket URL returns *Access Denied*; the CloudFront URL works.



REMEMBER

Deliverable: `docs/phase-6-ec2-s3-site.md`: architecture diagram, proof screenshots (Access Denied vs working site), and why each control matters. Delete the WAF web ACL and endpoints afterwards.

QUICK CHECK (answers at the back)

1. Why does an instance in a private subnet need VPC endpoints for Session Manager?
2. What does Origin Access Control protect against?
3. Which region must a CloudFront WAF web ACL be created in?

Interview question: "A developer made an S3 bucket public to host a website. How would you fix it without breaking the site?"

S4 Terraform: everything as code

7 – 20 Dec · Guide: G3 ch. 12 · Optional: AWS Cloud Practitioner exam about 18 Dec

> **Goal: rebuild S2 and S3 as code, so one command builds your environment and one command removes it.**

Why it matters for security

- Code is **reviewed** before it runs, and it's scannable (Checkov in B4).
- The same code gives the same result every time, so there's no forgotten click.
- `terraform destroy` is your cost-control button.

The four commands

Command	What it does
`terraform init`	Downloads the provider (the AWS plugin) and sets up state
`terraform plan`	Shows what *would* change. Always read it.
`terraform apply`	Makes the changes, after you type `yes`
`terraform destroy`	Removes everything this code created

Your first file

```
terraform {
  required_providers {
    aws = { source = "hashicorp/aws" }
  }
}
provider "aws" {
  region  = "eu-west-2"
  profile = "lab"
  default_tags { tags = { owner = "manuel", sprint = "S4" } }
}
resource "aws_vpc" "lab" {
  cidr_block = "10.40.0.0/16"
  tags       = { Name = "lab-vpc" }
}
resource "aws_subnet" "public_a" {
  vpc_id            = aws_vpc.lab.id     # a reference: Terraform works out the order
  cidr_block        = "10.40.1.0/24"
  availability_zone = "eu-west-2a"
}
```

Build steps

1. Install Terraform on Kali from HashiCorp's official instructions. Run `terraform -version`.
2. Recreate the S2 VPC (subnets, route tables, internet gateway), then the S3 static site.
3. Split it into a **module** (`modules/vpc`) with `variables.tf` and `outputs.tf`.

4. **Remote state:** keep the state file in an encrypted, versioned S3 bucket (with state locking), not on your laptop. State can hold secrets, so never commit it.
5. Add to `.gitignore`: `.terraform/`, `*.tfstate*`, `*.tfvars`.
6. **Timed drill:** `apply` → check → `destroy` within one session.

REMEMBER

Deliverable: `infra/aws/` in the repo with a README (what it builds, how to run it, how to destroy it) and `docs/phase-7-terraform.md`.

QUICK CHECK (answers at the back)

1. Why should you always read `terraform plan` before `apply`?
2. Why must the state file never go into Git?
3. What does `aws_vpc.lab.id` tell Terraform?

Interview question: "Why is infrastructure as code a security control and not just a convenience?"

S5 Holiday sprint (lighter)

21 Dec – 3 Jan · about 10 hours a week

> **Goal: rest, catch up and consolidate. Nothing new is required.**

- Finish anything left from S1–S4 and tidy the write-ups.
- **Solutions Architect – Associate study:** pick one course and one set of practice exams. Aim for about 30% of the course done by 3 January.
- Post a LinkedIn carousel: "Phase A, month 2: what I built in AWS" (redacted screenshots).
- Re-read your interview bank and answer five questions out loud.

REMEMBER

Missing a lighter sprint is fine. Skipping rest every sprint isn't.

S6 Serverless, databases + Cognito

4 – 17 Jan · Guide: G3 ch. 9

> **Goal: build an API with no servers to manage, protect it with a real login, and run a database that the internet can't reach.**

Part 1: a serverless API (all in Terraform)

1. **DynamoDB** table `notes` (on-demand billing).
2. **Lambda** function (Python) that reads and writes the table. Its **execution role** allows only `dynamodb:GetItem/PutItem/Query` on that one table: least privilege.
3. **API Gateway HTTP API** with routes `GET /notes` and `POST /notes` to the Lambda.
4. Test with `curl`, then read the Lambda's logs in CloudWatch.

Part 2: secure login with Cognito (video project 2)

1. **Cognito user pool:** a strong password policy, **MFA required** (authenticator app) and email verification.
2. Add a **JWT authorizer** to the HTTP API using the user pool, so requests without a valid token get `401`.
3. Prove both cases: no token → 401; valid token → data.

Part 3: RDS in a private subnet

- The smallest PostgreSQL instance, in private subnets, with **no public access**, encryption on, and a security group that allows 5432 **only from the Lambda's or app's security group**, not from an IP range.
- Store the master password in **Secrets Manager** (RDS can manage it for you). Never in code. **Destroy after the session.**

REMEMBER

Deliverable: `infra/aws/serverless-api/` + `docs/phase-8-serverless.md` with the 401 vs 200 proof.

QUICK CHECK (answers at the back)

1. What is a Lambda execution role, and why scope it to one table?
2. Why reference a security group instead of an IP range in the RDS rule?
3. What does the JWT authorizer check?

Interview question: "Who is responsible for patching in Lambda, you or AWS?"

S7 Scaling, containers + CloudTrail → Wazuh

18 – 31 Jan · Guides: G2 ch. 8–10; G3 ch. 13

> **Goal: run an app that survives a server dying, ship it as a container, and watch your AWS account from your own SIEM.**

Part 1: load balancer + Auto Scaling

- An **Application Load Balancer** in the public subnets; instances in the **private** subnets, in an **Auto Scaling group** (min 2, across two AZs), launched from a **launch template**.
- Instance security group: allow port 80 **only from the ALB's security group**.
- **Break it:** terminate one instance and watch the group replace it.

Part 2: containers on ECS Fargate

```
aws ecr create-repository --repository-name lab-app --image-scanning-configuration
scanOnPush=true
aws ecr get-login-password | docker login --username AWS \
  --password-stdin ACCOUNT_ID.dkr.ecr.eu-west-2.amazonaws.com
docker build -t lab-app . && docker tag lab-app ACCOUNT_ID.dkr.ecr.eu-west-
2.amazonaws.com/lab-app:v1
docker push ACCOUNT_ID.dkr.ecr.eu-west-2.amazonaws.com/lab-app:v1
```

Run it as an ECS Fargate service behind the ALB. Read the ECR scan results: that's your first container vulnerability report.

Part 3: CloudTrail → Wazuh (Case 005)

1. Create an IAM identity for Wazuh that can **only read the CloudTrail bucket** (reuse your S2 policy pattern).
2. Install the Python dependencies listed in the Wazuh docs ("Monitoring AWS-based services → Prerequisites") on the Wazuh server, and store the credentials in an AWS profile there.
3. Add the AWS module to `ossec.conf` and restart:

```
<wodle name="aws-s3">
  <disabled>no</disabled>
  <interval>10m</interval>
  <run_on_start>yes</run_on_start>
  <bucket type="cloudtrail">
    <name>YOUR-CLOUDTRAIL-BUCKET</name>
    <aws_profile>wazuh</aws_profile>
  </bucket>
</wodle>
```

4. Check that pfSense lets MGMT reach the internet on **HTTPS (443)**. The connection is outbound only.
5. **Generate activity in your own account:** a failed console login, creating an access key, changing a security group. Find each event in Wazuh (`data.aws.eventName`).
6. Write **Case 005**: "My home SIEM sees my cloud." Which events would you alert on, and why?

IN YOUR LAB

This is hybrid detection on a budget: one SIEM watching on-prem and cloud. It's the same idea as Sentinel in B8, and a good service for the startup.

REMEMBER

Deliverable: `docs/phase-9-cloudtrail-wazuh.md` + Case 005. Destroy the ALB, ASG and ECS service afterwards.

QUICK CHECK (answers at the back)

1. Why put instances in private subnets behind an ALB?
2. What does `scanOnPush=true` do?
3. Why give Wazuh its own read-only identity instead of your admin login?

Interview question: "Which CloudTrail events would you alert on first in a new AWS account?"

S8 The Azure sprint

1 – 14 Feb · Guide: G3 (Azure columns) · Optional: AZ-900

> **Goal: rebuild the basics in Azure so you're comfortable in both clouds, and see that the concepts carry over.**

Translate what you know

AWS (you know it)	Azure	Watch for
Account	Subscription (in a tenant)	Resource groups hold resources
IAM Identity Center / IAM	Entra ID + Azure RBAC	Identity (Entra) is separate from permissions on resources (RBAC)
VPC / subnet	VNet / subnet	Azure reserves 5 IPs per subnet, like AWS
Security group	NSG	NSGs attach to subnets *or* network interfaces
Session Manager	Azure Bastion	Bastion costs per hour
S3	Storage account (Blob)	Disable public blob access
IAM role for EC2	Managed identity	No secrets in code, same idea
CloudTrail	Activity Log	Sentinel reads it in B8

Build steps

1. **Azure for Students** with your school email. Turn on MFA and create a budget alert.
2. **Entra ID:** create two users and a group. Give the group `Reader` on one resource group, then test what a Reader can and can't do.
3. **Azure CLI on Kali:** `az login`, then `az account show`.
4. **Terraform (`azurerm` provider):** a resource group, a VNet with two subnets, an NSG, a small Linux VM with no public IP, and a storage account with public access disabled. The provider needs a `features {}` block and your subscription ID.
5. `terraform destroy`, then write a one-page **AWS ↔ Azure comparison** in your own words.

REMEMBER

Deliverable: `infra/azure/` + `docs/phase-10-azure.md` + the comparison note.

QUICK CHECK (answers at the back)

1. What's the difference between Entra ID and Azure RBAC?
2. What is the Azure equivalent of an EC2 instance role?
3. Where can an NSG be attached?

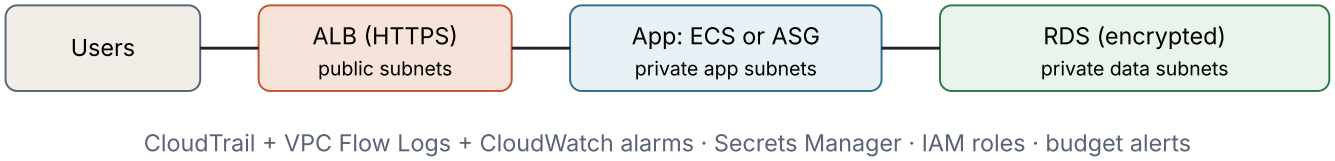
Interview question: "You know AWS well. How would you get productive in an Azure environment in your first week?"

S9 Phase A capstone

15 – 28 Feb · Book AWS Solutions Architect – Associate for early March

> **Goal: one project that proves "I can design, build, secure and destroy a real AWS application from code."**

The build: a 3-tier web app



CloudTrail + VPC Flow Logs + CloudWatch alarms · Secrets Manager · IAM roles · budget alerts

"Done when" checklist

- One `terraform apply` builds it all; one `destroy` removes it all.
- Three tiers in separate subnets; each security group allows traffic only from the tier above.
- No public IPs on the app or database; HTTPS at the load balancer.
- Secrets in Secrets Manager; least-privilege IAM roles; encryption at rest.
- Logging: CloudTrail, VPC Flow Logs and at least one CloudWatch alarm.
- An architecture diagram and a README: what, why, the security decisions, the cost per hour, and what you would add next.
- A 2–3 minute screen-recorded demo (good for LinkedIn and interviews).

Phase A checkpoint

Can you…	Yes?
Explain IAM policies, roles and the evaluation order?	☐
Design a VPC with public and private subnets from memory?	☐
Build and destroy an environment from Terraform?	☐
Explain how each tier is protected?	☐
Build the basics in Azure and map them to AWS?	☐
Show AWS activity in your SIEM?	☐

REMEMBER

If most boxes are ticked, you've met the February goal: comfortable building on AWS and Azure. Anything unticked becomes B1's first week.

Interview question: "Tell me about a cloud project you built. What were the security decisions, and what would you change?"

B Phase B: cloud security outline (Mar – Dec 2027)

> **Phase A taught you to build. Phase B teaches you to secure, monitor and respond. Each month gets its own detailed workbook pages when it starts.**

Month	Learn	Build / prove
Mar · Identity	Policy conditions, permission boundaries, Access Analyzer, Organizations and SCPs, Identity Center; Entra Conditional Access and PIM	A multi-account setup with SCPs that block risky actions (e.g. disabling CloudTrail)
Apr · Data	KMS and key policies, S3 security (Object Lock), Secrets Manager, Macie, ACM/TLS; Azure Key Vault	Customer-managed keys on S3 and RDS; a Macie finding on test data
May · Network + app	SG/NACL design, VPC endpoints, Flow Logs analysis, WAF rules, Shield	WAF blocks a test attack on your own site · **Security+**
Jun · Posture + DevSecOps (lighter)	AWS Config, Security Hub, CIS benchmarks, Prowler; Checkov; GitHub Actions	A pipeline that blocks insecure Terraform
Jul · Detection	CloudTrail deep dive, GuardDuty, Athena, metric filters; detection as code	5 tested detections in `detections/aws/`
Aug · Response	NIST IR lifecycle in the cloud, evidence preservation, Detective	GuardDuty → EventBridge → Lambda auto-response; 3 incident write-ups
Sep · Containers + serverless	Image scanning, task roles, Kubernetes security (k3s at home), Lambda hardening	A hardened container deployment with scanning in the pipeline
Oct · Azure + Sentinel	Defender for Cloud, Sentinel with the AWS connector, KQL	A multi-cloud SIEM with 3 KQL rules · **AWS Security Specialty**
Nov · Capstone	Putting it all together	Secure landing zone + app + logging + detections + auto-response + an end-to-end incident
Dec · Job sprint	CV, portfolio, interview practice	Applications going out; buffer for anything that slipped

The three incident simulations (August)

Scenario	Detect with	Respond by
Leaked access key	GuardDuty, CloudTrail (unusual API calls, new region)	Deactivate the key, review what it did, rotate, write up
Public S3 bucket	Config rule, Security Hub, Access Analyzer	Block Public Access, check access logs, assess data exposure
Compromised EC2	GuardDuty finding, VPC Flow Logs	Isolation security group, EBS snapshot for evidence, investigate, rebuild from code

REMEMBER

All simulations run in your own account, against your own resources. Use safe test methods such as GuardDuty's sample findings and deliberately misconfigured test resources.

R Resources

For	Free	Paid (optional)
AWS hands-on and theory	AWS Skill Builder (free tier), AWS documentation and Well-Architected Framework	A Solutions Architect Associate course from a well-reviewed instructor
AWS exam practice	Official AWS sample questions	One reputable practice-exam set
Terraform	HashiCorp's official tutorials (developer.hashicorp.com)	
Azure	Microsoft Learn (AZ-900 path); Azure for Students credit	
Cloud security	AWS Security Reference Architecture; AWS security blog; flaws.cloud and flaws2.cloud (legal practice challenges)	An AWS Security Specialty course (Phase B)
SOC thread	TryHackMe free rooms, LetsDefend free tier, CyberDefenders	TryHackMe subscription for SOC Level 1
Security+	Professor Messer's free videos	A practice-exam set

Pick one course per cert and finish it. Switching courses halfway is one of the biggest time wasters.

How to study faster

- **Build the same day you learn.** Video → console → write-up, in one evening.
- **Practice questions from week 1,** not at the end. Wrong answers show you what to study next.
- **Explain it out loud** (or to Claude). If you can't explain it simply, you don't know it yet.
- **One deliverable per sprint, pushed.** Unfinished builds don't count; small finished ones do.

A Answers to the quick checks

S1

1. The root user has unlimited power and can't be restricted by policies. If it's compromised, the whole account is lost. Use it only for a few billing and account tasks.
2. It accepts syslog only from pfSense's MGMT address (10.0.3.1), so other machines can't inject fake logs into the SIEM.
3. A region is a geographic area (e.g. London); an availability zone is one or more separate data centres inside a region. Spreading across AZs survives a data-centre failure.

S2

1. Its route table has a route for `0.0.0.0/0` to an internet gateway.
2. `ListBucket` acts on the bucket ARN; `GetObject` acts on object ARNs (`bucket/*`).
3. Replies to allowed traffic are automatically allowed back. You don't write return rules.

S3

1. It has no route to the internet, and the SSM agent must reach the Systems Manager service. The endpoints give it a private path.
2. Anyone reading the bucket directly. Only your CloudFront distribution can fetch objects, so WAF and HTTPS can't be bypassed.
3. `us-east-1` (N. Virginia), because CloudFront is a global service.

S4

1. The plan shows exactly what will be created, changed or **destroyed**. Reading it catches mistakes before they happen.
2. State can contain secrets and the full map of your infrastructure. Keep it in an encrypted, private backend.
3. It's a reference to the VPC's ID, which also tells Terraform to create the VPC before the subnet.

S6

1. The identity the function uses to call other AWS services. Scoping it to one table limits the damage if the function is abused (least privilege).
2. It allows only resources in that security group, even if their IPs change, and nothing else in the range.
3. That the token was issued by your user pool, is signed correctly, is not expired, and is for the expected audience.

S7

1. The instances aren't reachable directly from the internet. Only the load balancer is exposed, and it forwards only the allowed port.
2. ECR scans each image for known vulnerabilities when it's pushed.
3. Least privilege: if the Wazuh server or its keys are compromised, the attacker can read only that bucket.

S8

1. Entra ID manages identities (users, groups, sign-in, MFA). Azure RBAC decides what those identities can do to Azure resources.
2. A managed identity.
3. To a subnet or to a network interface (or both).